Comprendre et utiliser les dossiers du kit DEUKSIL

Ce document explique, **dossier par dossier**, ce qu'il contient, **à quoi il sert**, **quand** l'utiliser et **comment**, avec des commandes complètes. Il complète le guide (`GUIDE_EQUIPE_BACKEND.md`), qui donne l'ordre des étapes.

Le principe en 30 secondes

DEUKSIL est fait de **deux parties** :

1. **Le site** (ce que voit l'utilisateur) : `index.html` , `css/` , `js/` , `img/` . Des fichiers à publier chez un hébergeur web. Rien à installer.
2. **Le serveur Supabase** (comptes, données, fichiers, sécurité) : tout ce qui est dans `supabase/` . Il est installé **dans votre projet Supabase** grâce aux scripts fournis.

Les scripts sont **déjà écrits et testés**. L'équipe ne les modifie pas : elle les **lance** sur son projet Supabase, avec **ses** valeurs (adresse du projet, clés), rangées une fois pour toutes dans un fichier `.env` .

Avant tout : le fichier `.env` (une seule fois)

Toutes les commandes ont besoin des valeurs de votre projet. Plutôt que de les recopier à chaque commande, on les met dans un fichier `.env` .

1. Créer le fichier à partir du modèle :

```
cp .env.example .env
```

2. Le remplir avec un éditeur de texte (valeurs du projet **de test** : Dashboard Supabase → Project Settings → API et → Database) :

```
SUPABASE_URL=https://abcdefghijklmnop.supabase.co
SUPABASE_ANON_KEY=eyJhbGciOi...            # clé « anon » (publique)
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOi...    # clé « service_role » (SECRÈTE)
DATABASE_URL=postgresql://postgres:MOT-DE-
PASSE@db.abcdefghijklmnop.supabase.co:5432/postgres
DEUKSIL_ORIGINES=https://test.votre-domaine.sn
APP_URL=https://test.votre-domaine.sn/
MOT_DE_PASSE_TEST=Choisi-Par-Equipe-2026
DEUKSIL_TEST_AUTORISE=1
```

3. Le charger dans le terminal, au début de chaque séance de travail :

```
set -a; source .env; set +a
```

Ensuite, toutes les commandes de ce document fonctionnent **telles quelles**, sans rien recopier.

> `.env` contient des secrets : il est déjà exclu de Git (`.gitignore`). Ne jamais l'envoyer par message ni le joindre à un ticket. Un fichier `.env` par projet (test, production) : on peut les nommer `.env.test` et `.env.prod` et charger le bon (`source .env.test`).

Carte des dossiers

Dossier / fichier	Contenu	Utilisé quand	Qui
`supabase/migrations/`	8 scripts SQL : toute la base	une fois par projet, puis à chaque évolution	backend
`supabase/functions/gerer-acces/`	fonction serveur de création de comptes	une fois par projet, puis si elle change	backend
`supabase/templates/`	2 modèles d'e-mails	une fois par projet (copier-coller)	backend
`supabase/config.toml`	réglages de référence	base locale ; référence pour le tableau de bord	backend
`supabase/outils/`	2 outils : schéma pour le site ; import du jeu de test	à chaque changement de schéma ; projet de test	backend
`supabase/tests-local/`	tests de sécurité, jeu de test, mesures, réplique	projet de test	backend / QA
`supabase/sauvegarde/`	sauvegarde et restauration	chaque jour/semaine ; essai trimestriel	backend / exploitation
`tests/`	tests navigateur + fiche de recette	projet de test	QA
`js/00-config.js`	réglages publics du site	une fois par projet (test, prod)	frontend / backend
`docs/`	rapports (sécurité, performances, index)	lecture	tous

---

1. `supabase/migrations/` — la base de données

Ce que c'est. Huit fichiers SQL numérotés. Ensemble, ils créent **toute** la base : les 32 tables, les règles d'accès (qui voit quoi), les règles métier (numéros de factures, rappels, etc.), les index de performance.

Fichier	Contenu en clair
`…0100_base.sql`	extensions et outils de base
`…0200_tables.sql`	les 32 tables (comptes, biens, baux, loyers, factures…)
`…0300_comptes_et_droits.sql`	création automatique du profil à l'inscription, protection des comptes

Fichier	Contenu en clair
`…0400_rls.sql`	les règles d'accès : chacun ne voit que ses données
`…0500_fonctions_api.sql`	fonctions appelées par le site (annuaire, notifications…)
`…0600_stockage.sql`	dossiers de fichiers (photos, avatars, documents privés)
`…0700_regles_et_rappels.sql`	numérotation, rappels quotidiens, règles de visites, de messages…
`…0800_performances.sql`	optimisations et index

Comment l'utiliser (étape 5 du guide) — on ne lance **pas** les fichiers un par un, la commande Supabase s'en charge, dans l'ordre :

```
supabase link --project-ref abcdefghijklmnop     # une fois : relier le dossier au projet
supabase db push                                 # applique les 8 fichiers dans l'ordre
```

Vérifier dans l'éditeur SQL du tableau de bord : `select count(*) from pg_tables where schemaname = 'public';` doit répondre **32**.

Règle importante : un fichier déjà appliqué ne se modifie **jamais**. Pour faire évoluer la base, on crée un nouveau fichier (`supabase migration new nom_du_changement`), puis `supabase db push` .

---

2. `supabase/functions/gerer-acces/` — la fonction serveur

Ce que c'est. Un petit programme qui tourne chez Supabase. Il crée les comptes qu'un propriétaire ou une agence ouvre pour quelqu'un d'autre (membre d'équipe, locataire, propriétaire mandant). Il est le seul à utiliser la clé secrète `service_role` , **côté serveur**, jamais dans le navigateur.

Comment l'utiliser (étape 7) :

```
supabase functions deploy gerer-acces
supabase secrets set DEUKSIL_ORIGINES=$DEUKSIL_ORIGINES
```

Vérifier : `curl -i -X POST $SUPABASE_URL/functions/v1/gerer-acces` doit répondre **401** (« connexion requise ») : c'est normal, la fonction refuse les inconnus.

---

3. `supabase/templates/` — les e-mails

Ce que c'est. Deux pages HTML : l'e-mail de confirmation d'inscription et celui de mot de passe oublié. Chacun affiche un **code à 6 chiffres**.

Comment l'utiliser (étape 4) : pas de commande. Dans le tableau de bord → Authentication → Email Templates : * « Confirm signup » : coller le contenu de `confirmation.html` ; objet : `Confirmez votre adresse DEUKSIL` ; * « Reset password » : coller `recovery.html` ; objet : `Votre code DEUKSIL` .

---

4. `supabase/config.toml` — les réglages de référence

Ce que c'est. La liste des réglages Supabase recommandés (sécurité des mots de passe, e-mails, API…).

Comment l'utiliser : sur un projet hébergé, ces réglages se font **à la main** dans le tableau de bord, en suivant la `FICHE_PARAMETRES.md` (rubriques B et C). Le fichier sert aussi si l'équipe veut une base Supabase locale (`supabase start` , nécessite Docker).

---

5. `supabase/outils/` — deux outils

Outil	À quoi il sert	Quand
`importer-json.js`	charge le **jeu de données de test** (29 comptes fictifs, biens, baux…)	projet de **test** uniquement, avant les tests
`generer-schema-front.js`	met à jour `js/02c-schema.js` , la description des tables utilisée par le site	après chaque nouvelle migration qui touche les tables

Commandes (après `source .env` , projet de test) :

```
npm install @supabase/supabase-js@2.45.4 pg            # une fois
MOT_DE_PASSE=$MOT_DE_PASSE_TEST node supabase/outils/importer-json.js \
  supabase/tests-local/jeu-de-test.json supabase/tests-local/comptes-test.json

node supabase/outils/generer-schema-front.js           # après un changement de tables
```

> **Jamais `importer-json.js` sur la production** : il crée des comptes de test.

---

6. `supabase/tests-local/` — tests et données de test

> 📦 **Les outils de test ne sont pas dans ce dossier** : le projet est livré **vide** (aucun compte, aucun bien, aucune donnée). Les tests automatiques et le jeu de données de test (`supabase/tests-local/` , `tests/*.js`) sont dans **`DEUKSIL_OUTILS_DE_TEST.zip`** , à décompresser à la racine du projet **uniquement sur le poste qui teste le projet Supabase de TEST**. Ne jamais publier ces fichiers sur l'hébergement.

Élément	Rôle
`securite-rls.test.js`	**65 tests de sécurité** : se connecte avec 11 vrais comptes et vérifie que chacun ne voit et ne modifie que ce qu'il doit
`jeu-de-test.json`	données fictives (adresses `@deuksil.test`)
`comptes-test.json`	liste des comptes de test (rôle, e-mail), produite par l'import
`perf/`	mesures de performance (facultatif, jamais en production)

Élément	Rôle
`replique/`	copie locale de Supabase utilisée pour le développement de ce kit (facultatif)
`resultats/`	résultats des derniers tests

Commande (étape 9) :

```
node supabase/tests-local/securite-rls.test.js
```

Attendu : `65/65 tests réussis`.

---

7. `tests/` — tests du site et recette

Élément	Rôle	Commande
`acces-comptes.js`	ouverture d'accès locataire / équipe / mandant	`node tests/acces-comptes.js` → 14/14
`sonde-pages.js`	ouvre toutes les pages des 9 profils, cherche les erreurs	`node tests/sonde-pages.js` → « OK » partout
`ecritures-auto.js`	vérifie que le site n'écrit rien sans action de l'utilisateur	`node tests/ecritures-auto.js` → aucune ligne
`xss-sonde.js`	tente d'injecter du code dans les données (**modifie la base : en dernier**)	`node tests/xss-sonde.js` → « Aucune exécution »
`RECETTE_MANUELLE_FORMULAIRES.md`	fiche à cocher : inscription, code e-mail, mot de passe oublié, photos, contact	à la main, 15 min

Prérequis une fois : `npm install playwright pg && npx playwright install chromium`.

---

8. `supabase/sauvegarde/` — sauvegarder et restaurer

Ce que c'est. Deux scripts : * `sauvegarder.sh` copie **toute la base** (comptes compris) **et tous les fichiers** (photos, documents) dans un dossier daté ; * `restaurer.sh` remet une sauvegarde dans un projet **neuf** (il refuse un projet qui contient déjà des comptes, pour protéger la production).

Sauvegarder (après `source .env`) :

```
cd supabase/sauvegarde
npm install                    # une fois
./sauvegarder.sh               # crée sauvegardes/AAAAMMJJ-HHMM/
```

Le dossier obtenu contient des **données personnelles** : le chiffrer (`gpg -c`) et le ranger hors du projet.

Restaurer (dans un projet neuf où les migrations ont été appliquées) :

```
DATABASE_URL_CIBLE="postgresql://postgres:…@db.<ref-neuf>.supabase.co:5432/postgres" \
SUPABASE_URL="https://<ref-neuf>.supabase.co" \
SUPABASE_SERVICE_ROLE_KEY="<clé service_role du projet neuf>" \
CONFIRMER=oui ./restaurer.sh sauvegardes/AAAAMMJJ-HHMM
```

Attendu à la fin : `VÉRIFICATION OK : mêmes comptages qu'à la sauvegarde.`

---

9. `js/00-config.js` — brancher le site sur Supabase

Ce que c'est. Le **seul** fichier du site à modifier. Il dit au site à quel projet Supabase parler.

Comment : l'ouvrir dans un éditeur et remplir :

```
supabaseUrl: 'https://abcdefghijklmnop.supabase.co',
supabaseAnonKey: 'eyJhbGciOi...',     // clé ANON uniquement — jamais service_role
googleActif: false,
siteUrl: 'https://test.votre-domaine.sn',
editeur: { raisonSociale: '…', ninea: '…', /* mentions légales */ }
```

Puis publier les fichiers du site chez l'hébergeur.

---

10. `docs/` — les rapports

Document	À lire si…
`AUDIT_SECURITE.md`	on veut savoir ce qui a été sécurisé, testé, et ce qui reste à surveiller
`PERFORMANCES.md`	on veut les mesures avant / après et les limites connues
`INDEX.md`	on se demande pourquoi un index existe

Résumé : une séance type sur le projet de test

```
cp .env.example .env.test           # une fois, puis remplir
set -a; source .env.test; set +a    # à chaque séance

supabase link --project-ref <ref>   # une fois
supabase db push                    # base
supabase functions deploy gerer-acces
supabase secrets set DEUKSIL_ORIGINES=$DEUKSIL_ORIGINES
# → régler e-mails et adresse du site dans le tableau de bord (FICHE_PARAMETRES)
# → remplir js/00-config.js et publier le site de test

MOT_DE_PASSE=$MOT_DE_PASSE_TEST node supabase/outils/importer-json.js \
  supabase/tests-local/jeu-de-test.json supabase/tests-local/comptes-test.json
node supabase/tests-local/securite-rls.test.js
node tests/acces-comptes.js
```

```
node tests/sonde-pages.js
node tests/ecritures-auto.js
# → fiche RECETTE_MANUELLE_FORMULAIRES.md à la main
node tests/xss-sonde.js             # en dernier

cd supabase/sauvegarde && npm install && ./sauvegarder.sh
```

Questions fréquentes

Faut-il modifier des scripts ? Non. Seuls `.env` (vos valeurs) et `js/00-config.js` (site) se remplissent.

Que faire si une commande échoue ? Lire le message, consulter `README_SUPABASE.md` §18 (erreurs fréquentes), ne pas passer à l'étape suivante.

Peut-on lancer les tests sur la production ? Non. Les tests et l'import créent ou modifient des données : projet de test uniquement.

Où sont les mots de passe et clés ? Dans le tableau de bord Supabase et dans `.env` (local, jamais partagé). Nulle part ailleurs.